

Document number: P2817R0
Date: 2023-03-05
Audience: SG21
Reply-to: Andrzej Krzemiński <akrzemi1 at gmail dot com>

The idea behind the contracts MVP

The purpose of this document is to explain the plan that we in SG21 call *contracts MVP*.

Executive summary

The idea of *contracts MVP* is based on two observations. First, P0542-contracts ([\[P0542R5\]](#)) were too big to be added within a three- or even six-year time frame. Second, a far smaller subset of it exists that could satisfy many groups of users, that could be standardized faster, and still leave the room for adding the reminder of P0542-contracts (or superior alternatives) in subsequent iterations.

The proposed feature is small: annotations for preconditions, postconditions and assertions, plus the C-assert execution model: either don't evaluate these predicates, or evaluate and stop the program if they fail.

Our goal is to encourage SG21 members to focus on the details of contract annotations first, so that we may have a sound and stable minimum incremental change in the Working Draft in the direction of the full contract support framework. In order to gather support from people that need the contract support framework to provide more capabilities, rather than adding them just yet, we try to demonstrate how the Contracts MVP allows to add the required additional features on top in a backwards compatible manner. We try to demonstrate that we are closing as little design doors as possible.

The longer version

The idea emerged after the contract support framework proposed in [\[P0542R5\]](#) has been removed from the Working Draft just before C++20 was finalized. There was a number of controversies and misunderstandings that prevented reaching consensus, and in consequence the only thing to do was to revert to the "status quo", which meant getting nothing. The really sad thing about it is that the core functionality of contract annotations was not controversial at all. It was:

- the standardized way for programmers to declare contract annotations as C++ expressions,
- compiler to verify if these expressions are syntactically and type-system-wise correct,
- other programmers and reviewers to be able to see these annotations,
- tools to make arbitrary use of these annotations, beyond the scope of the C++ Standard.

A good illustration of this observation was a cry by one of WG21 members, "just standardize the syntax without any semantics". Of course, one needs *some* semantics associated with the syntax in order for any tool to make a use of these annotations. However, for the MVP (Minimum Viable Product), we try to define the minimum required semantics to make the syntax of contract annotations meaningful. We avoid specifying how you can control the compiler to make these contract annotations do different things in different builds.

The procedure in WG21 is that if a consensus about the details of a given feature cannot be reached, the feature is not added and we are left with the status quo: the last state of the Working Draft that WG21 had consensus on. P0542-contracts ([\[P0542R5\]](#)) were considered a single feature. The lack of consensus over a number of "controls" (how continuation works, are levels enough or do we want "labels" or "roles" or "semantics", when can contract conditions be "assumed" by the compiler) meant that the entire P0542-contracts had to be removed. In other words, the status quo for the contract "controls" was no contracts.

The goal for the *Contracts MVP* is to have a better status quo when contract "controls" are discussed: the status quo that is not just nothing, but something of value, even if this value is small. This is based on the observation that some use cases for contracts do not need any "controls" at all: they only need contract annotations to be present in the code. We do not want these use cases to be kept hostage by the different and incompatible designs of the controls, and by the disagreements between the involved parties.

What are the use cases that do not require any controls? First, a library/component designer has a tool for describing contract predicates in a single standardized way. If they use this ability, the sole act of putting the preconditions and postconditions into function declarations (and compiler checking if they are type-system-correct) may trigger the author to think harder about the design and highlight a flaw even before the program is compiled and run. This has the potential to prevent bugs, rather than detect the added bugs.

Second use case is IDEs. If they can see and understand the contract predicates (due to the standardized syntax and the usage of C++ expressions), they can give this information to the programmers in a clever way. For instance, when a programmer writes an expression that calls a function with a declared precondition, the IDE can display the precondition but instead of using the function parameter names, it can use the names of objects passed to the function as arguments.

Third use case is code analyzers that can perform static analysis or inject some instrumentation into the compiled binaries. This also requires controlling nothing in the compiler. Many useful checks in static analyzers cannot be written today because we are missing contract annotations in the code. These tools require programmers to put contract annotations in a standardized formats. We need programmers to start putting these annotations now.

Fourth, programmers that use libraries/components with contract annotations can see the declared preconditions and postconditions, and maybe this is the first time they will understand the contract and the semantics of the library/component they use. This again has the potential to prevent bugs in the first place, rather than detecting the planned bug.

Fifth, your favorite compiler can assign, even if in a non-conforming mode, arbitrary meaning to these contract annotations, such as performing runtime checks to log and/or abort when they fail.

Finally, the important use case offered by the MVP, as we see it, is that it offers a foundation for evolution. It enables compilers to implement extensions on top of the contract annotation syntax, which would be a good basis for the future standardization, based on implementation, deployment and usage experience. The MVP has been carefully focused not to preclude any evolution path.

Importantly, the plan is *not* to deliver nothing more than the MVP for C++26. It is rather to secure a better status quo. Once this is done, other contract-related features, such as "controls", can be attempted to be added still to C++26 without the risk of losing the entire contract support from the Working Draft. Admittedly, one might be annoyed with the name "minimum *viable* product". It boils down to what one considers viable. Maybe the choice of name is not optimal, and name "secure the minimum contracts status quo" would be a more precise term.

Our plan is to build on top of the work already done by the authors [\[P0542R5\]](#). Because they put a lot of work into hammering the details of contract annotations, we believe that by taking a subset of P0542-contracts we have a chance of succeeding in getting something into C++26. But even this subset has a number of issues that need to be addressed. The past heated discussions about contract annotation "semantics" and "controls" were so absorbing that they shadowed the second level of bugs in the design for contract annotations. For instance, during the work on the *Contracts MVP* we were able to pin down a number of design details missing from P0542-contracts; for instance,

- the semantics of contract annotations in virtual functions in classes with multiple bases or bases being template parameters,
- whether contract annotations are in the immediate context,
- availability in freestanding implementations.

(For details see [\[P2521R3\]](#).) Our goal is to encourage SG21 members to focus on the details of contract annotations first, so that we may have a sound and stable minimum incremental change in the Working Draft in the direction of the full contract support framework. In order to gather support from people that need the contract support framework to provide more capabilities, rather than adding them just yet, we try to demonstrate how the Contracts MVP allows to add the required additional features on top in a backwards compatible manner. We try to demonstrate that we are closing as little design doors as possible. For instance, [\[P2182R0\]](#), which introduced the notion of *contracts MVP*, indicates how the continuation after failed run-time check, contract levels, global toggles and literal semantics can be added on top of the MVP while preserving backwards compatibility. [\[P2176R0\]](#) discusses how inexpressible predicates can be added on top. [\[P2784R0\]](#) shows how the property of not halting the program upon detected contract violation can be added on top of the MVP.

We are aware that we do not address some important use cases, including these that motivated [\[P0542R5\]](#). By this we are not trying to imply that they are less important. It is just that we see how to address the use cases that we selected faster, and obtain the feedback from the real life usage faster, while we can still work on re-adding the remaining use cases. Our goal is *not* to stop after delivering the Contracts MVP.

Technically, we could have defined an even thinner scope. It has been proposed to provide only precondition annotations without postcondition annotations, as the latter are more problematic (see [\[P2521R3\]](#)). However, we refuse to do this as we feel this would cross the boundary between something that makes a coherent whole and a random, meaningless subset. Postconditions are necessary in certain form of program analysis where the tool can match preconditions of functions that consume values against the postconditions of functions that produce these values. This has been confirmed by an SG21 poll on 2021-12-14:

Poll: Postconditions should be in the MVP at this time.

SF	F	N	A	SA
1	7	3	4	1

In a similar vein, one could question the need to provide translation mode *Eval_and_abort* in the MVP. However, we feel it is necessary in order to define the range of allowed program behaviors. Clearly, at some point, we will need more than one translation mode, and this means the same source code can be used to generate different executable code, with different semantics. In order to define the range of allowed behaviors resulting from contract annotations, we provide the two extreme points:

- *No_eval*, which means that compiler must not evaluate the predicates, has the following implications:
 1. Putting contract annotations is not a guarantee that the program will stop upon the detected violation (because it might be compiled in *No_eval* mode). You still have to make sure that the code is structured so that the contract annotations are never violated.
 2. The predicates may not be evaluated (because the program might be compiled in *No_eval* mode), don't count on their effects (side effects, or premature termination).
- *Eval_and_abort*, which means that the compiler must guarantee that the execution of the function does not continue after the contract predicate returns `false`, has the following implications:
 1. The predicates in your contract annotations may be evaluated, so be careful what you put inside.
 2. In the worst case your program may be terminated if a predicate evaluates to `false`, so make sure you structure your program so that the predicates never evaluate to `false`.

A compiler could offer more modes, even if as a non-conformant extension, but they will fall somewhere between these two extreme points. This is why we need to define two translation modes in the MVP, and don't require more.

It should also be mentioned that the MVP does not propose contract declarations to be added to the Standard Library.

One benefit of the present MVP approach is that we were able to better expose and document the design space and challenges for the things like side effects in the contract predicates (see [\[P2570R2\]](#) and [\[P2680R1\]](#)), as well

as the mutation of the non-reference function parameters referenced in postconditions (see [\[P2521R3\]](#)).

One could be tempted to ask, "give me this additional switch", or "give me this additional feature", or otherwise some users will not be able to use it to full extent, or it will be "not viable" for someone. We understand that the MVP will not please all the users. However, the risk of taking the route of adding more and more is that we will not be able to deliver anything in the timely fashion. We would end up repeating the history of "C++0x" where users had to wait like eight years for the new standard, or repeating the history of P0542-contracts that already aimed at being small but still needed to be ripped out due to too many controversial issues. Instead, we try to demonstrate how the MVP does not stand in a way of adding these features in the future, and ask for understanding. Our goal is to deliver a foundation in C++26 rather than something bigger in an indeterminate future.

Acknowledgments

We are grateful to the authors of [\[P0542R5\]](#) — G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup — for their work on the contract support framework. The goal of the MVP is basically to reintroduce P0542-contracts, or something very similar, albeit in stages.

Joshua Berne, John Spicer and Timur Doumler reviewed the draft of this paper and offered improvements.

References

- [\[P0542R5\]](#) — G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "Support for contract based programming in C++" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0542r5.html>).
- [\[P2114R0\]](#) — Joshua Berne, Ryan McDougall, Andrzej Krzemiński, "Minimal Contract Use Cases" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2114r0.html>).
- [\[P2176R0\]](#) — Andrzej Krzemiński, "A different take on inexpressible conditions" (<https://isocpp.org/files/papers/P2176R0.html>).
- [\[P2182R0\]](#) — Andrzej Krzemiński, Ryan McDougall, Joshua Berne, "Contract Support: Defining the Minimum Viable Feature Set" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2182r0.html>).
- [\[P2698R0\]](#) — Bjarne Stroustrup, "Unconditional termination is a serious problem" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2698r0.pdf>).
- [\[P2521R3\]](#) — Gašper Ažman, Joshua Berne, Bronek Kozicki, Andrzej Krzemiński, Ryan McDougall, Caleb Sunstrum, "Contract support — Record of SG21 consensus" (<https://isocpp.org/files/papers/P2521R3.html>).
- [\[P2570R2\]](#) — Andrzej Krzemiński, "Contract predicates that are not predicates" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2570r2.html>).
- [\[P2680R1\]](#) — Gabriel Dos Reis, "Contracts for C++: Prioritizing Safety" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2680r1.pdf>).
- [\[P2784R0\]](#) — Andrzej Krzemiński, "Not halting the program after detected contract violation" (<https://isocpp.org/files/papers/P2784R0.html>).